

GAP ANALYSIS & IMPROVEMENT ROADMAP

KOD HARNESS

A line-level audit of the kod coding-agent harness against the observations of the Typesafe Coding Agent feedback doc — KV-cache economics, tool-schema overhead, compaction, subagent state passing, and the typesafe-native features kod is closest to shipping — with ranked, code-level fixes.

Engineering review · harness core audit
Evidence cited from 116,700-line workspace
dump

SEPTEMBER 2026 · V1.0

Table of Contents

1. Executive Summary	1
2. Observation-to-Kod Gap Map	2
3. P0 - Make the KV Cache Real: Accounting, Transcript Caching, Filter Stability	4
3.1 What the audit found	4
3.2 Fix 1: parse and price cache tokens	5
3.3 Fix 2: breakpoint the transcript	7
3.4 Fix 3: tool-filter hysteresis	7
4. P1 - Switch-Penalty-Aware Routing	8
5. P2 - Meta-Attention Context Engine and Transcript Rehydration	10
5.1 From FIFO truncation to scored fidelity	10
5.2 Rehydration: load relevant old state on the fly	12
6. P3 - Tool Search and Lazy Schemas	13
7. P4 - Conditional AGENTS.md	15
8. P5-P8 - Subagent Briefs, Background Review, Security Routing, Batteries	17
8.1 P5: typed subagent briefs and reports	17
8.2 P6: background read-only jobs and cross-model review	17
8.3 P7: security-aware routing	18
8.4 P8: search batteries and relevance-truncated output	19
9. Phased Roadmap and Engine Hygiene	20

1. Executive Summary

This report maps the observations in **Thoughts on a Typesafe Coding Agent** (the "feedback doc") onto the kod codebase, using a line-level audit of the 116,714-line workspace dump. The overall verdict is positive: kod already implements roughly half of the doc's typesafe-native vision, often with unusually good engineering. Skills ship genuine progressive disclosure (a capped name-and-description inventory plus full bodies only on match), Jev already provides section-level query-aware budget reweighting, the policy engine is a pure queryable decision function with provenance, memory is compaction-immune by construction, and the PromptPlan/SystemSegment chain with golden-prefix tests treats the cacheable prefix as a first-class resource. The doc's critique of "while loops with a small number of tools" undersells how much deliberate machinery kod wraps around that loop.

The headline finding: kod does world-class cache engineering for the system prompt, and leaves the two largest cache surfaces dark. The growing transcript never carries a `cache_control` breakpoint, cache tokens are never parsed, and the per-turn Jev tool filter mutates the tools array that sits inside the cached prefix. On Anthropic, every turn re-bills the entire conversation at full input price, and kod cannot even measure the waste because `cache_read` and `cache_creation` tokens are dropped on the floor.

This reframes the feedback doc's sharpest economic claim (Thing 1: routing to a cheaper model and back costs more than staying) for kod specifically. Kod re-resolves the endpoint chain on every single turn with zero accounting for KV-cache state, so the session happily bounces local and cloud endpoints. That penalty is currently invisible in kod's cost tracking, because with no transcript caching and no cache token parsing, full-price re-processing happens on every turn regardless. Fix the cache plumbing first (P0), and switch-aware routing (P1) becomes measurable and implementable in days rather than being guesswork.

Move	Why it matters	Effort
P0. Cache token accounting + transcript breakpoint + tool-filter hysteresis	Makes the doc's Thing 1 measurable; cuts input billing on long sessions by roughly an order of magnitude at 0.1x cache-read pricing; protects the whole prefix from silent invalidation.	Days
P1. CacheLedger switch-penalty routing	Prices the re-processing penalty before the engine hops endpoints; routing only happens when the economics actually win.	Days
P4. Conditional AGENTS.md loader	The doc's conditional-system-messages idea is a completely open lane in kod; biggest missing typesafe-native feature per unit of work.	Days
P2. Meta-attention context engine + transcript rehydration	The doc's flagship idea; kod's typed chunks and session_log.jsonl already contain the raw material, so it is integration work rather than a rewrite.	1-2 weeks
P3. tool_search and lazy schemas	Completes the skills-style progressive disclosure for tools and MCP; removes the per-turn filter pressure that P0 hysteresis works around.	Days

Table 1: Top five moves, ranked by expected leverage on cost and agent quality.

Everything in this document cites dump.txt line numbers (for example engine.rs 27969 means dump lines 21997-35209, the engine.rs file). Section 2 gives the full observation-by-observation gap map. Sections 3 through 7 cover the four P0/P1/P2 movements and the two feature lanes with Rust reference implementations. Section 8 compresses four further ideas to signature level, and section 9 closes with a phased roadmap and the small hygiene fixes the audit surfaced along the way.

2. Observation-to-Kod Gap Map

The feedback doc makes six "tyranny of the KV cache" observations and proposes a set of typesafe-native features. The table below checks each one against kod's actual code, with the primary evidence line-referenced. Verdicts are Shipped (kod already does this or something better), Partial (a foundation exists but the idea is not realized), or Missing (no prior art in the codebase). Three observations land as Missing and one of them, routing economics, is actively costing money today. Two of the doc's requests are already substantially answered by kod's existing design, which the doc's author may not have assumed.

Observation	Kod today (evidence)	Verdict
1. Routing does not work; switching models re-processes context	Model chain re-resolved every turn (Jev pick, by_task table, fallback list); no cache-state or cost signal anywhere in resolve_chain_for_task (engine.rs 27508); TokenUsage has only prompt/completion fields (kod-provider types.rs 62105), so the penalty is unmeasurable.	Missing
2. Tool calling is a weird tradeoff (up-front schemas)	Schemas are hand-written and lean: about 5.8 KB / 1.4k tokens compact for 23 tools; the prompt carries one-line descriptions only (engine.rs 29916); but the Jev category filter rewrites the tools array per turn, and the tools array sits inside the Anthropic cached prefix before system blocks (wire.rs 57680-57854).	Partial
3. Compaction exists; query-aware compression would beat it	No summarization at all: FIFO caps of 40 turns, 4k chars per turn, 32k char budget (engine.rs 22909-22945); decisions log and plan are re-rendered every turn so durable state outlives the FIFO; Jev reweights section budgets per query (engine.rs 26492) but nothing scores individual history chunks.	Partial
4. Subagents are meh; state passing is the problem	Swarm subagents receive a one-paragraph brief plus a role preamble, no parent-transcript slice; engine-global transcript interleaving is documented as a known weakness (swarm_runner.rs 45122); worktrees, expected_writes glob claims and re-plan-on-collision already solve the write side.	Partial
5. Restarting exists; load relevant state on the fly	Restart restores display prose and seeds text turns only; tool calls and results are dropped (the repo's own review notes this and proposes restore_transcript_from_log, dump 102343, unimplemented); session_log.rs already records every ToolCall, ToolOutcome, PolicyDecision and Cost as JSONL.	Missing
6. Batteries are not included	25 lean built-in tools plus MCP; search is regex over the ignore walker (no ripgrep, no AST); no first-party integrations with compression or structural-search tooling.	Partial
Meta-attention (per-chunk context scoring)	Seed exists at section granularity: reallocate_with_jev re-splits history/skills/memory/repomap budget shares per query (engine.rs 26492) and filters memory entries individually (26616); ChatMessage already carries id, pinned, token_count metadata, so per-chunk scoring slots in cleanly.	Partial
First-principled Skills/MCP (snippet, then dump schema on demand)	Skills already do two-layer progressive disclosure: an inventory capped at 60 name-and-description lines, full instructions only on lexical+Jev match (router.rs 41420-41521); MCP tools get a startup tools/list snapshot that is never refreshed and no schema-dump path.	Partial
Conditional system messages / AGENTS.md	Zero support: no AGENTS.md, CLAUDE.md, or per-directory instruction file is read anywhere in the 116k lines; skills under .kod/skills are the only user-authored project context channel.	Missing

Observation	Kod today (evidence)	Verdict
Recursive language models / explicit state	Good bones: DecisionRecord log, plan re-rendering, swarm blackboard, write-glob claims; no general typed variable store that survives transcript aging.	Partial
Fancier summarization (heatmap grep output)	grep caps at 500 matches and 1 KB per entry, search_files at 200 matches (tools.rs 74228); truncation is mechanical, never relevance-ranked against the query.	Missing
Security-aware routing	Trust levels and taint escalation exist (trust.rs 96648), ReadProtection globs for secrets exist (policy.rs 14648); neither feeds model selection.	Missing
Background processing / cross-model review	Only hourly memory consolidation and a startup check baseline run in the background; the swarm REVIEWER role can use a different endpoint via routing.swarm, but there is no background job runner and no automatic cross-model review pipeline.	Missing

Table 2: Feedback-doc observations versus kod's audited state (dump.txt line refs).

Read as a whole, the map says kod's architecture already chose the typesafe-native direction in several places: typed prompt segments, typed memory entries, typed policy decisions with provenance, and typed subagent write claims. That is exactly why the remaining gaps are cheap to close. The next five sections work through them in descending order of leverage, each with a reference implementation that follows kod's existing patterns (fail-open Jev decisions, byte-stable prefixes, JSONL audit entries) so the changes review like kod code rather than a foreign transplant.

3. P0 - Make the KV Cache Real: Accounting, Transcript Caching, Filter Stability

3.1 What the audit found

On the wire, Anthropic receives tools first, then system blocks, then messages. Kod places exactly one ephemeral cache_control marker on the last cacheable system segment (wire.rs 57680-57697), which caches the identity and repo-map head but nothing else. The messages array never carries a marker (wire.rs 57716-57822), so the transcript - the part that actually grows turn over turn - is re-billed at full input price on every call. On OpenAI-compatible providers kod relies on accidental byte-stability with no explicit prompt_cache_key and no accounting either. Meanwhile TokenUsage records only prompt_tokens and completion_tokens (types.rs 62105): cache_read_input_tokens and cache_creation_input_tokens are parsed nowhere, cost.rs bills cache reads at the full input rate, and the golden-prefix test pins the system text but not the wire-level tools array. The practical consequence is that a 100k-token transcript costs roughly ten times what it should on Anthropic's 0.1x cache-read pricing,

and kod's /debug tokens and budget screens over-report spend while under-reporting cache health.

The third defect is the subtle one. Because tools precede system on the wire, the single system-level breakpoint implicitly covers the tool definitions too. Kod's Jev category filter (engine.rs 27164) and MCP top-3 trim (25942) rewrite the tools array per turn; every time the filter flips a category, the entire cached prefix - tools, system, repo map - goes cold. The registry already sorts tools by name specifically to protect prefix stability (registry.rs 71931), so the per-turn filter is working against one of kod's own stated invariants.

3.2 Fix 1: parse and price cache tokens

Extend TokenUsage with the two Anthropic cache fields, parse them in both the non-stream usage object and the SSE message_start and message_delta events, and teach ModelPricing the cache rate tiers with safe defaults. When a provider does not report the fields they stay zero and cost math degrades to today's behavior, so this change is backward compatible across providers. Cost entries in session_log.jsonl and the /debug tokens screen gain a cache hit-rate readout, which is the prerequisite instrument for everything in section 4.

kod-provider/src/types.rs and kod-core/src/cost.rs

```

#[derive(Debug, Clone, Default, Serialize, Deserialize)]
pub struct TokenUsage {
    pub prompt_tokens: u64,
    pub completion_tokens: u64,
    pub total_tokens: u64,
    // NEW: cache visibility. Zero for providers that do not report it.
    #[serde(default)]
    pub cache_read_tokens: u64, // served from KV cache, ~0.1x price
    #[serde(default)]
    pub cache_creation_tokens: u64, // written to KV cache, ~1.25x price
}

impl TokenUsage {
    /// Input tokens actually processed at full price this call.
    pub fn uncached_input_tokens(&self;) -> u64 {
        self.prompt_tokens
            .saturating_sub(self.cache_read_tokens + self.cache_creation_tokens)
    }
    pub fn cache_hit_rate(&self;) -> f64 {
        if self.prompt_tokens == 0 { 0.0 }
        else { self.cache_read_tokens as f64 / self.prompt_tokens as f64 }
    }
}

pub struct ModelPricing {
    pub input_per_mtok_usd: f64,
    pub output_per_mtok_usd: f64,
    pub cache_read_per_mtok_usd: f64, // NEW, default 0.1 * input
    pub cache_write_per_mtok_usd: f64, // NEW, default 1.25 * input
}

impl ModelPricing {
    pub fn cost_usd(&self;, u: &TokenUsage;) -> f64 {
        let m = 1_000_000.0;
        (u.uncached_input_tokens() as f64 / m) * self.input_per_mtok_usd
        + (u.cache_read_tokens as f64 / m) * self.cache_read_per_mtok_usd
        + (u.cache_creation_tokens as f64 / m) * self.cache_write_per_mtok_usd
        + (u.completion_tokens as f64 / m) * self.output_per_mtok_usd
    }
}

```

kod-provider-anthropic/src/wire.rs - usage parsing

```

fn parse_usage(v: &serde::json::Value) -> TokenUsage {
    let get = |k: &str;| v.get(k).and_then(Value::as_u64).unwrap_or(0);
    let cache_read = get("cache_read_input_tokens");
    let cache_write = get("cache_creation_input_tokens");
    TokenUsage {
        // Anthropic's input_tokens EXCLUDES the cache fields; kod's
        // prompt_tokens is the total billed-and-processed window.
        prompt_tokens: get("input_tokens") + cache_read + cache_write,
        completion_tokens: get("output_tokens"),
        total_tokens: 0,
        cache_read_tokens: cache_read,
        cache_creation_tokens: cache_write,
    }
}
// SSE: message_start carries the input side once; message_delta carries
// cumulative output_tokens - merge it, do not replace the struct.

```

3.3 Fix 2: breakpoint the transcript

Anthropic caches everything up to and including the block that carries the marker, and allows multiple breakpoints per request. Kod should add a second marker on the last message of the messages array, gated on the provider declaring explicit caching. The first call after the change pays the 1.25x write premium once; every subsequent turn reads the transcript back at 0.1x. Steering still works exactly as today: `apply_steers` mutates the pending user text before the next call, and a mutated message simply misses the cache from that point forward - a price event, never a correctness event. The same gate should refuse to add the marker when a tool filter change is pending, which pairs with Fix 3.

kod-provider-anthropic/src/wire.rs - messages_array

```
fn messages_array(prompt: &CompletionRequest;) -> Vec {
    let mut out = Vec::with_capacity(prompt.messages.len());
    let n = prompt.messages.len();
    for (i, m) in prompt.messages.iter().enumerate() {
        let mut msg = message_to_wire(m);
        // NEW: incremental transcript caching. Only the FINAL message carries
        // the marker; earlier markers would fragment the cache
        // entry pool (Anthropic allows 4 breakpoints; kod uses 2).
        if i + 1 == n
            && prompt.cache_gate == CacheGate::Enabled // set by engine, see 3.4
        {
            if let Some(blocks) = msg.get_mut("content").and_then(Value::as_array_mut) {
                if let Some(last) = blocks.last_mut() {
                    last["cache_control"] = json!({ "type": "ephemeral" });
                }
            }
        }
        out.push(msg);
    }
    out
}
```

3.4 Fix 3: tool-filter hysteresis

Keep the built-in tool list byte-stable within a session phase and stop re-filtering it every turn. Concretely: compute the Jev category filter only when the classified task signature changes and the previous set has survived a hysteresis window; restrict per-turn semantic trimming to MCP tools, which are unstable by nature and sit behind the `mcp: namespace`; and hold the marker gate (Fix 2) for one round after any change does land. The engine already has the right hook points: definitions are computed once per turn at `engine.rs 28026` and attached per request at `29850`, so the filter state is a small addition next to them.

kod-core/src/engine.rs - stable tool filter

```

pub struct ToolFilterState {
    enabled_categories: BTreeSet, // BTree = deterministic order
    task_signature: String,
    last_changed_turn: u64,
    min_stable_turns: u64, // hysteresis window, e.g. 3
}

impl ToolFilterState {
    /// Refilter only when the task type actually changed AND the previous
    /// tool set had at least `min_stable_turns` to pay for its cache write.
    pub fn should_refilter(&self, turn: u64, task_sig: &str;) -> bool {
        let changed = task_sig != self.task_signature;
        let seasoned = turn.saturating_sub(self.last_changed_turn) >=
            self.min_stable_turns;
        changed && seasoned
    }

    pub fn commit(&mut self, turn: u64, task_sig: String,
        cats: BTreeSet) {
        self.last_changed_turn = turn;
        self.task_signature = task_sig;
        self.enabled_categories = cats;
    }
}

// Invariant to add to the golden tests: for a fixed task signature the
// serialized CompletionRequest.tools must be byte-identical across turns,
// exactly like the system prefix today.

```

4. P1 - Switch-Penalty-Aware Routing

The feedback doc's Thing 1 is an economics proof: routing a task to a cheaper model and back re-processes the context on the expensive model, and with its example proportions pure Opus comes out at two thirds the cost of the round trip. Kod is the perfect illustration of why this matters, because `resolve_chain_for_task` (engine.rs 27508-27556) rebuilds the endpoint chain from the turn's classified task type on every turn, and `pick_endpoint_with_jev` (26277) can pick a different endpoint each time. Nothing in that path knows whether the target endpoint's KV cache is warm, what a switch would cost, or what the previous turn cost. Meanwhile `cost.rs` is a pure accumulator: it records and caps spend, and its only routing adjacent behavior is to stop or ask when a cap trips.

The fix is a small ledger that rides alongside the existing chain resolution. For each endpoint it remembers the fingerprint of the last request head it served, the last observed cache token counts, and the last turn it was used. When the chain resolves, a candidate that is not warm pays a projected re-processing penalty derived from the current transcript size and the candidate's cache price tiers; a candidate that is warm pays nothing. The switch happens only when the projected saving of the cheaper endpoint clears the penalty by a configurable margin, which implements exactly the hysteresis the doc asks for ("make educated cost-aware choices about re-using the existing KV cache"). Note the beautiful simplification that arrives with subagents: their contexts are fresh and small, so per-capability routing inside swarms (which kod already has at 27496) is the cache-safe place to be aggressive with cheap models. The ledger

makes the main loop equally deliberate.

kod-core/src/cache_ledger.rs (new), wired into resolve_chain_for_task

```
#[derive(Default, Clone)]
pub struct EndpointCacheState {
    pub prefix_fingerprint: u64,    // FNV-1a of last served request head
    pub cached_tokens: u64,        // last cache_read_tokens observed
    pub last_used_turn: u64,
}

#[derive(Default)]
pub struct CacheLedger {
    states: HashMap, // endpoint id -> state
    sticky: Option,  // endpoint we are warm on
}

impl CacheLedger {
    /// The doc's Thing 1, priced. Cold target => full transcript re-processed
    /// at cache-write + cache-read tiers on first use there.
    pub fn switch_penalty_usd(&self, target: &str, head_fingerprint: u64,
                             pricing: &ModelPricing, transcript_tokens: u64) -> f64 {
        let warm = self.states.get(target)
            .map(|s| s.prefix_fingerprint == head_fingerprint)
            .unwrap_or(false);
        if warm { return 0.0; }
        let m = 1_000_000.0;
        (transcript_tokens as f64 / m)
            * (pricing.cache_write_per_mtok_usd + pricing.cache_read_per_mtok_usd)
    }

    /// Resolve: keep the by_task chain as the ORDER, but gate the hop.
    pub fn gate(&self, preferred: &Endpoint, fallback: &Endpoint,
                per_turn_saving_usd: f64, penalty_usd: f64) -> &Endpoint; {
        // 1.5x margin: do not hop for marginal wins (hysteresis).
        if penalty_usd == 0.0 || per_turn_saving_usd > penalty_usd * 1.5 {
            preferred
        } else {
            fallback // == sticky warm endpoint when available
        }
    }

    pub fn observe(&mut self, turn: u64, id: &str, fingerprint: u64,
                  usage: &TokenUsage) {
        let s = self.states.entry(id.to_string()).or_default();
        s.prefix_fingerprint = fingerprint;
        s.cached_tokens = usage.cache_read_tokens;
        s.last_used_turn = turn;
        self.sticky = Some(id.to_string());
    }
}

// engine.rs integration: build head_fingerprint from the rendered PromptPlan
// cacheable prefix + sorted tool schema bytes; call ledger.observe() where
// record_cost() runs today (engine.rs 28728) so the ledger learns for free.
```

Two design notes. First, the ledger must observe, not assume: the fingerprint is computed from bytes kod already renders, and the cache token counts come from the P0 parsing, so the ledger's world model is grounded in provider truth rather than heuristics. Second, expose it: a /cache command (alongside /debug tokens) showing per-endpoint hit rates, warm states and the last penalties the gate declined turns an invisible economic layer into something a power user can reason about, which is very much in the spirit of kod's existing /limits and policy explain surfaces.

5. P2 - Meta-Attention Context Engine and Transcript Rehydration

5.1 From FIFO truncation to scored fidelity

The doc's flagship proposal is meta-attention: stop treating context as static, re-score every chunk against the current query, and render each chunk at one of four fidelities - do not show it, show a small summary, show a longer summary, or show the whole thing. Kod is closer to this than any codebase the doc's author might assume: `reallocate_with_jev` already asks three yes/no questions per turn (needs repomap, needs history, needs skills) and redistributes the section budget while preserving the total (`engine.rs 26492`), and memory entries are individually Jev-filtered (`26616`). What is missing is the same treatment one level down, at the level of individual transcript chunks. Today's alternative is recency FIFO: 40 turns, 4k characters per turn, oldest dropped (`engine.rs 22909-22945, 32165`), with pinned turns, the decisions log and the plan deliberately immune.

The data model is ready. `ChatMessage` already carries a stable id, role, `tool_calls`, `MessageMetadata` with pinned and `token_count` (`kod-types message.rs 95081`), and `kod-memory` already ships a hybrid BM25-lite plus vector plus recency scorer (`retrieval.rs 55144`) with an explicitly deliberate brute-force vector index (`vector_index.rs 56231`). The context engine below reuses those exact primitives so the scoring behaves like the rest of kod: deterministic, testable, and fail-open. Jev remains the semantic backstop, batched so that scoring the whole history costs one yes/no-style call rather than one per chunk.

kod-core/src/context_engine.rs (new)

```

#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Debug)]
pub enum Fidelity { Omit, Stub, Digest, Full } // the doc's four levels

#[derive(Clone, Copy)]
pub enum ChunkKind { UserTurn, AssistantTurn, ToolCall, ToolResult }

pub struct ContextChunk {
    pub id: ChunkId, // stable across turns (from ChatMessage.id)
    pub kind: ChunkKind,
    pub raw: Arc,
    pub turn_index: u32,
    pub pinned: bool, // user-pinned turns: always Full
    pub digest: OnceCell, // lazy 2-3 sentence summary, computed once
    pub tokens: u32,
}

pub trait ChunkScorer: Send + Sync {
    /// How much does the CURRENT query need this chunk, and at what fidelity?
    fn score(&self, chunk: &ContextChunk, query: &Query;) -> Fidelity;
}

/// Deterministic default scorer. Mirrors kod-memory's retrieval weights
/// (semantic/keyword 0.6/0.3, recency half-life 14d) so behavior is familiar.
pub struct LexicalScorer {
    pub index: KeywordIndex, // BM25-lite over chunk text, reused from
    pub half_life_turns: f64, // kod-memory/src/retrieval.rs 55144
}

impl ChunkScorer for LexicalScorer {
    fn score(&self, c: &ContextChunk, q: &Query;) -> Fidelity {
        if c.pinned { return Fidelity::Full; }
        let kw = self.index.bm25(&c.id, q.terms());
        let recency = (-c.age_turns() as f64 / self.half_life_turns).exp2();
        // Tool errors and the most recent assistant turn stay legible.
        let hot = matches!(c.kind, ChunkKind::ToolResult) && c.mentions_error();
        match (kw, recency) {
            (k, _) if k > 2.5 => Fidelity::Full,
            (k, _) if k > 1.2 => Fidelity::Digest,
            (k, r) if k > 0.4 || r > 0.5 || hot => Fidelity::Stub,
            _ => Fidelity::Omit,
        }
    }
}

/// Optional Jev pass: one batched call scores the Stub/Digest boundary
/// candidates only (fail-open, like every other Jev decision in kod).
pub struct JevScorer { inner: LexicalScorer, client: JevClient }
impl ChunkScorer for JevScorer { /* refine boundary chunks in one batch */ }

```

kod-core/src/engine.rs - render_history_for, upgraded

```
fn render_scored(chunks: &[ContextChunk], q: &Query;,
                 scorer: &dyn; ChunkScorer, budget: CharBudget) -> String {
    let mut out = String::new();
    for c in chunks {
        match scorer.score(c, q) {
            Fidelity::Omit => {} // chronological order preserved
            Fidelity::Stub => out.push_str(&stub;_line(c)),
            Fidelity::Digest => out.push_str(c.digest_or_compute()),
            Fidelity::Full => out.push_str(&c.raw;),
        }
    }
    // Stub is 1 line ("tool read_file crates/web/x.ts - 340 lines, 12:0x"),
    // Digest is 2-3 sentences computed once and cached on the chunk.
    truncate_preserving_fulls(out, budget)
}
// Invariants to pin in characterization tests: pinned turns and the decisions
// block are always Full; Omit never removes the last assistant turn; total
// never exceeds the (Jev-reweighted) Allocation.history share.
```

5.2 Rehydration: load relevant old state on the fly

The doc's Thing 5 observes that restarting exists because agent state corrupts, and suggests loading relevant old state on demand instead of replaying everything. Kod's own production review agrees at dump line 102343: after a crash, sessions restart with rendered prose only, and it proposes `restore_transcript_from_log` - unimplemented. The raw material is already perfect: `session_log.rs` appends typed `ToolCall`, `ToolOutcome`, `PolicyDecision`, `Cost` and `MemoryRetrieval` entries as crash-tolerant JSONL (43583-44462), and kod replay already reads them. The rehydrator below turns that log back into `ContextChunks` that start COLD and get scored per query by the P2 engine, which is precisely "load up all old relevant state on the fly" - restart stops being amnesia and becomes lazy loading.

kod-core/src/rehydrate.rs (new)

```
pub fn rehydrate_transcript(log: &Path;, cwd_fingerprint: u64)
    -> anyhow::Result<>
{
    let reader = session_log::Reader::open(log)?; // drops truncated tail (43955)
    let mut chunks = Vec::new();
    for entry in reader {
        match entry {
            SessionEntry::ToolCall(tc) => chunks.push(chunk_from_tool_call(tc)),
            SessionEntry::ToolOutcome(o) => chunks.push(chunk_from_outcome(o)),
            // MemoryRetrieval / Cost / PolicyDecision stay audit-only, as today.
            _ => {}
        }
    }
    Ok(chunks) // all COLD: rendered only when the scorer says so
}

// TUI load_session (app.rs 84148) then does:
// 1. seed prose turns as today (they render as Full for one turn),
// 2. rehydrate cold chunks from ~/.kod/session/.jsonl,
// 3. hand everything to render_scored(), which silently pulls back only
//    what the new query needs - the tool state survives restart again.
```

6. P3 - Tool Search and Lazy Schemas

The doc's Thing 2 wants something between nothing and everything: small snippets about what is available, the ability to dump full schemas when needed, and no context corruption. Kod's skills system already implements exactly this pattern for instructions; tools still ship all schemas on every request. To be fair to kod, its tool block is unusually lean - roughly 1.4k tokens of compact JSON for 23 hand-written schemas, with one-line descriptions in the prompt instead of full schemas (engine.rs 29916) - so the waste is modest today. The problem scales with MCP: every connected server's tools arrive with whatever schemas they carry, the startup tools/list snapshot is never refreshed (mcp_adapters.rs 37713), and the Jev top-3 trim that manages this (25942) is precisely the cache-hostile per-turn mutation P0 works around. The durable fix is to give tools the skills treatment.

kod-tools/src/tool_search.rs (new) + tiered registry

```

pub enum ToolTier {
    A, // always full schema on the wire: the <= 10 core tools, byte-stable
    B, // one-liner in "## Tool use"; full schema retrievable on demand
}

pub struct ToolSearchTool { registry: Arc }

#[async_trait]
impl Tool for ToolSearchTool {
    fn definition(&self;) -> ToolDefinition {
        ToolDefinition {
            name: "tool_search".into(),
            description: "Search the available-but-not-loaded tool catalog by \
                        name or purpose. Returns FULL parameter schemas for the \
                        matches so you can call them on your next turn."
                .into(),
            category: ToolCategory::System,
            parameters_schema: json!({
                "type": "object",
                "properties": {
                    "query": { "type": "string" },
                    "limit": { "type": "integer", "minimum": 1, "maximum": 8 }
                },
                "required": ["query"]
            }),
            permissions: ToolPermissions::READ_ONLY, // Allow in every preset
            trust_level: TrustLevel::System,
        }
    }

    async fn execute(&self;, params: Value, _ctx: &ToolContext;) -> ToolOutput {
        let q = params["query"].as_str().unwrap_or_default();
        let hits = self.registry.search_inventory(q, 4); // BM25 over
                                                    // name+description

        ToolOutput::json(serde_json::json!({
            "tools": hits, // name, description, FULL parameters_schema
            "note": "schemas injected into your next request; call normally"
        })))
    }
}

// Registry changes (registry.rs): register_with_tier(); get_definitions_for_llm
// returns tier_a ++ tier_b_promoted, where promoted = tools the model fetched
// via tool_search this turn. Prompt keeps listing EVERY tool as a one-liner
// (name: description), so the model always knows what exists - the doc's
// "small snippets" layer - while the wire stays small and stable.
// MCP: tool_search queries the static snapshot AND triggers tools/list
// refresh for stale servers, replacing the per-turn top-3 trim entirely.

```

This closes the loop with P0: once Tier B tools are discovered on demand, the per-turn Jev category filter can be retired rather than made sticky, because the wire tool set only changes when the model itself asks it to. The change also de-risks MCP growth: adding a hundred tools stops costing a hundred schemas and starts costing a hundred one-liners, which is the doc's "hundreds of tools and thousands of docs built-in" endgame at kod's existing level of prompt hygiene.

7. P4 - Conditional AGENTS.md

The doc's conditional-system-messages idea - is it front-end? load the style guide; is it this subdirectory? load its footguns file - has zero prior art in kod. Nothing reads AGENTS.md or CLAUDE.md anywhere in the workspace; the identity prompt is a hardcoded string plus skills, memory and grounding. That makes this the highest value-per-day lane in the plan, and kod already contains every mechanism the feature needs: globset with gitignore semantics from the policy engine (policy.rs 15235), a directory-shadowing loader pattern in the skills loader (loader.rs 62661), the TaskType classification to key conditions on (router.rs 40295), and a volatile prompt slot that re-renders every turn so the feature is prompt-cache neutral by construction.

kod-config/src/instructions.rs (new)

```

#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum When {
    Always,
    Path(String),      // glob, gitignore semantics via policy.rs globset
    TaskType(String),  // Simple | CodeModification | Debugging | Research | ...
    Lang(String),       // rust | ts | python - from repomap file stats
}

#[derive(Debug, Clone)]
pub struct InstructionSection {
    pub when: When,
    pub source: PathBuf,    // which AGENTS.md it came from (for /debug output)
    pub body: String,
}

/// AGENTS.md with conditional fences:
///
/// ## Frontend footguns
/// - never edit tailwind.config.ts by hand; regenerate via `pnpm gen`
///
pub fn parse_agents_md(src: &str, from: &Path;) -> Vec {
    let mut out = Vec::new();
    let mut current = When::Always;
    let mut buf = String::new();
    for line in src.lines() {
        if let Some(w) = parse_open_fence(line) { current = w; buf.clear(); }
        else if is_close_fence(line) {
            out.push(InstructionSection { when: current,
                                         source: from.into(),
                                         body: std::mem::take(&mut; buf) });
            current = When::Always;
        }
        else { buf.push_str(line); buf.push('\n'); }
    }
    if !buf.trim().is_empty() {
        out.push(InstructionSection { when: current, source: from.into(), body: buf });
    }
    out
}

/// Root -> leaf chain, deeper files shadow shallower ones by section name,
/// mirroring the skills loader' shard rule (loader.rs 62661-62691).
pub fn load_chain(cwd: &Path;, repo_root: &Path;) -> Vec { /* */ }

// Router integration (router.rs build_prompt_with_budget): render only the
// sections whose `when` matches this turn's TaskResponse + touched paths as
// `## Project instructions` inside the VOLATILE suffix. They are
// compaction-immune by the same mechanism as plan and decisions: re-rendered
// from disk every turn, never dependent on transcript survival.

```

Two properties make this feel native rather than bolted on. First, the evaluation is deterministic and inspectable: a kod instructions explain command can print each section, its guard, and why it fired this turn, matching the provenance culture of policy explain and the JSONL audit. Second, conditions compose with the existing classification rather than replacing it - a Debugging turn in crates/web automatically gets both the debugging preamble kod already builds and the web footguns file the user wrote, which is exactly the division of labor the doc describes between harness conditioning and user memory.

8. P5-P8 - Subagent Briefs, Background Review, Security Routing, Batteries

8.1 P5: typed subagent briefs and reports

The doc blames subagent mediocrity on state passing: what context goes in, what comes back. Kod's swarm proves the write half is solved - expected_writes glob claims enforced at the tool boundary, collision re-planning, git worktree isolation - while the read half is a one-paragraph brief plus a static role preamble, with results concatenated post-hoc and never merged into the parent transcript (swarm_runner.rs 45633-46848). The fix is two typed records and two assemblers, reusing the P2 scorer to select parent context.

kod-swarm/src/brief.rs (new)

```
pub struct ContextBrief {
    pub goal: String,                // subtask description
    pub constraints: Vec,            // role preamble, quotas
    pub relevant_decisions: Vec,    // scored via LexicalScorer
    pub file_digests: Vec,          // path + 3-line summary
    pub repomap_slice: String,      // PageRank slice filtered
                                    // to subtask files
    pub expected_writes: Vec,       // existing glob claims
    pub token_budget: u32,
}

pub struct SubagentReport {
    pub summary: String,
    pub facts_learned: Vec,         // -> parent decisions log (Approach)
    pub files_touched: Vec,         // -> existing conflict check (46811)
    pub decisions_proposed: Vec,    // -> parent Jev decision extraction
    pub open_questions: Vec,       // -> parent steers queue (31738)
}

// swarm_runner.rs: brief = assemble_brief(&parent; chunks, subtask) using the
// P2 scorer instead of hand-picked slices; on join, merge_report() routes each
// field to the durable store it belongs to. Parent transcript stays clean;
// nothing is interleaved (per-agent keys swarm:{id} already exist).
```

8.2 P6: background read-only jobs and cross-model review

The doc's background-processing appendix describes read-only background agents - cross-model review, background evals, ELI5 docs - as extensions of the normal workflow, and notes the synergy: if finding relevant information is expensive, sharing that work across background tasks makes them cheap. Kod has fragments: REVIEWER and RESEARCHER role preambles that say "do not fix what you find", per-capability endpoint routing, a blackboard for shared findings, and steers as a delivery channel. What is missing is a runner that makes background a first-class, enforced state rather than a prompt-level suggestion.

kod-core/src/background.rs (new)

```

pub struct BackgroundJobRunner {
    jobs: DashMap,
    sem: Arc,           // cap concurrent background agents
}

impl BackgroundJobRunner {
    /// Read-only is ENFORCED at the tool layer, not the prompt: fork a child
    /// engine on the `read-only` preset (policy.rs 15132) intersected with an
    /// explicit whitelist [read_file, grep, search_files, file_info, git_status,
    /// git_diff, lsp_*, check]. No write tool exists to misuse.
    pub fn spawn_review(&self, parent: &KodEngine, subject: TurnId,
        endpoint: ModelRef) -> JobId {
        let child = parent.fork_read_only(READONLY_TOOLS);
        let digest = parent.render_last_turn(subject, Fidelity::Digest); // P2
        let prompt = format!("{REVIEW_PREAMBLE}\n\n{digest}");
        self.spawn(Job { kind: JobKind::Review { subject }, child, prompt,
            endpoint }) // cross-model: pick an endpoint DIFFERENT
                       // from the one that produced the turn,
                       // via [llm.routing.swarm] reviewer row
    }
}

// Results land on the blackboard and as an optional steer("reviewer notes: ..")
// the user can accept or ignore; /jobs and /jobs surface progress. The
// TUI keeps accepting input throughout - the swarm run's blocking UX (45103)
// was the only thing making "background" impossible before this.

```

8.3 P7: security-aware routing

The doc notes that much-cheaper models are a data-policy decision, not just a cost decision, and proposes file-type-aware routing policies. Kod has the trust machinery (ToolTrusted taint, ReadProtection secret globs at policy.rs 14648) and the routing hook (resolve_chain_for_task); the missing piece is thirty lines that connect them. Sensitivity comes from the paths a query or plan actually touches, and the endpoint registry gains a trust tier from config.

kod-core/src/sensitivity.rs (new) + endpoint gate

```
pub enum Sensitivity { Public, Internal, Sensitive }

pub fn classify(paths: &[PathBuf], policy: &PolicyEngine;) -> Sensitivity {
    if paths.iter().any(|p| policy.read_protected_matches(p)) // .env/.ssh/*.pem
        || paths.iter().any(|p| policy.denied_matches(p)) {
        Sensitivity::Sensitive
    } else if paths.is_empty() {
        Sensitivity::Public
    } else {
        Sensitivity::Internal
    }
}

// engine.rs resolve_chain_for_task, before the Jev pick:
// chain.retain(|e| match sensitivity {
//     Sensitivity::Sensitive => e.trust == TrustTier::Trusted,
//     Sensitivity::Internal  => e.trust >= TrustTier::Standard,
//     Sensitivity::Public    => true,
// });
// config.toml: [llm.endpoints.] trust = "trusted" | "standard" | "untrusted"
// Empty chain after retain => fall back to the trusted default and log a
// SessionEntry::ModelFallback, consistent with existing fallback auditing.
```

8.4 P8: search batteries and relevance-truncated output

The doc's appendix cites trajectory analysis suggesting reading and searching dominate agent token spend, and proposes loading better batteries with first-party prompts. Kod's search is competent but purely mechanical: regex over the ignore walker with hard caps (tools.rs 74228), no ripgrep, no structural search. Two upgrades fit kod's style: a ripgrep backend that spawns `rg --json` and falls back to the current walker when `rg` is absent, and relevance-window truncation that scores matches against the query instead of clipping at an arbitrary byte - the doc's "heatmap of what part of this grep output is relevant", achieved with lexical scoring rather than an LLM call.

kod-tools/src/search.rs - backend trait + heatmap truncation

```
#[async_trait]
pub trait SearchBackend: Send + Sync {
    async fn search(&self; q: &GrepQuery;) -> Result;
}
pub struct RegexWalker; // today's backend, kept as fallback
pub struct Ripgrep { rg: PathBuf } // spawn `rg --json -S`; if rg is
// missing, delegate to RegexWalker

/// Keep the matches that MATTER for this query, with their context windows.
pub fn heatmap_truncate(m: GrepMatches, q: &Query; cap_lines: usize) -> String {
    let mut scored: Vec<_> = m.entries.iter()
        .map(|e| (relevance(e, q), e)) // term overlap + file recency
        .collect();
    scored.sort_by(|a, b| b.0.cmp(&a.0;).then(a.1.path.cmp(&b.1.path;)));
    let mut out = String::new();
    for (_, e) in scored.iter().take_while(|(s, _)| *s > 0)
        .take(cap_lines / CTX_PER_MATCH) {
        out.push_str(&e.render;_with_context(2));
    }
    out.push_str(&format!("... {} more matches in {} files \
        (filtered by relevance to the query)",
        m.total - out_matches(&m;), m.files.len()));
    out
}
```

9. Phased Roadmap and Engine Hygiene

The plan compresses into three phases that respect dependency order: P0 makes cache economics measurable, P1 and the routing gate depend on that instrumentation, and the meta-attention engine underwrites the brief assembler and background digests. Each item is scoped to kod's existing extension points, so none of them requires a migration or a breaking config change; the only cross-cutting test duty is extending the golden/characterization suites to pin the new invariants (byte-stable tools for a fixed task signature, pinned-and-decisions-always-Full, rehydrated chunks rendering identically to their pre-restart selves).

Item	Impact	Effort	Phase and dependency
P0 cache accounting (tokens + pricing)	High	S	Week 1. Unblocks P1; independent of everything else.
P0 transcript breakpoint	High	S	Week 1. Gate behind P0 parsing + Fix 3 hysteresis.
P0 tool-filter hysteresis	High	S	Week 1. Protects prefix while P3 matures; golden test addendum.
P1 CacheLedger + routing gate	High	M	Month 1. Needs P0 observation data; /cache command alongside.
P4 conditional AGENTS.md	High	M	Month 1. Fully independent; ships user-visible value fastest.

Item	Impact	Effort	Phase and dependency
P2 meta-attention engine	High	L	Month 1-2. Scorer behind trait; Jev batch pass second.
P2 rehydration from JSONL	High	M	Month 2. Consumes P2 chunks; closes the dump-102343 TODO.
P3 tool_search + tiers	Medium	M	Month 2. Retires the Jev category filter when done.
P5 typed briefs/reports	Medium	M	Quarter. Reuses P2 scorer inside swarm_runner.
P6 background review jobs	Medium	M	Quarter. Uses P5 digests; /jobs TUI surface.
P7 sensitivity routing gate	Medium	S	Quarter. Thirty-line gate + config field + audit entry.
P8 ripgrep/ast-grep + heatmap	Medium	M	Quarter. Backend trait keeps RegexWalker as fallback.

Table 3: Effort-impact matrix and phasing for all recommendations.

The audit also surfaced small hygiene fixes worth batching into the first week because each is a one-liner to a few lines with outsized correctness value. None of them changes behavior contracts; they remove quiet waste and wrong metadata that the bigger features above would otherwise build on.

- `prompt_allocation` re-reads and re-parses the config file from disk on every turn (`engine.rs` 27409). Hold Arc behind a `tokio::watch` and reload on the config watcher event only.
- `OpenAICompatProvider` does not override `capabilities()`, so it inherits `conservative()` with `prompt_cache` `None` and `streaming_tools` `false` even though the stream path emits live tool text and the server prefix-caches (the repo's own review flags this at dump 102014). Override it and claim `PromptCacheKind::Automatic`.
- The legacy non-streaming summary path serializes tool results with Rust Debug formatting (`{:?}`) at `engine.rs` 28416. Route it through the structured render path so summaries stop consuming debug-escape bytes.
- No endpoint circuit breaker exists: a chronically failing primary is retried as primary every turn because the chain is rebuilt fresh. Add `EndpointHealth` with three-strikes cooldown of 60 seconds, skipped while cooling.
- `list_models` returns `Vec` and `Anthropic` returns an empty vector by choice (`wire.rs` 56880). Return per-model metadata (`context_window`, `pricing`) so `budget.rs` allocates against the selected model's true window instead of a global default.
- `providers.mdx` still says the Anthropic provider is not implemented while `build_provider` constructs it (`provider_setup.rs` 38910). Docs lagging code is a small thing that compounds in a harness this subtle.

The through-line across all nine sections is the same one the feedback doc preaches: being explicit about state is what turns a while loop into a system. Kod has been more explicit than most about prompts, policy

and memory; cache warmth, transcript chunks and subagent context are the three states it still treats implicitly, and they happen to be the three the doc predicts will matter most. The codebase's own habits - typed segments, golden tests, fail-open Jev decisions, JSONL audit - are precisely the tools needed to make them explicit, which is why every proposal here reads as an extension of kod rather than a criticism of it.